

Практическая работа 17. Создание приложения с адаптивным макетом

1. Введение В предыдущем коделабе вы начали преобразовывать приложение Reply в адаптивное, используя классы размеров окна и реализуя динамическую навигацию. Эти функции являются важной основой и первым шагом к созданию приложений для всех размеров экрана. Если вы пропустили кодельаб «Создание адаптивного приложения с динамической навигацией», настоятельно рекомендуем вам вернуться и начать с него.

В этом кодебалете вы будете развивать изученную концепцию для дальнейшей реализации адаптивной верстки в вашем приложении. Адаптивный макет, который вы будете реализовывать, является частью канонических макетов - набора часто используемых шаблонов для больших экранов. Вы также узнаете о дополнительных инструментах и методах тестирования, которые помогут вам быстро создавать надежные приложения.

Предварительные условия Завершение коделаба «Создание адаптивного приложения с динамической навигацией» Знаком с программированием на Kotlin, включая классы, функции и условия Знаком с классами ViewModel Знаком с функциями Composable Опыт создания макетов с помощью Jetpack Compose Опыт запуска приложений на устройстве или эмуляторе Опыт использования API WindowSizeClass Что вы узнаете Как создать адаптивный макет с видом списка с помощью Jetpack Compose Как создавать предварительные просмотры для разных размеров экрана Как тестировать код для разных размеров экрана Что вы будете создавать Вы продолжите обновление приложения Reply, чтобы оно было адаптивным для всех размеров экрана. Готовое приложение будет выглядеть следующим образом:

Что вам понадобится Компьютер с доступом в Интернет, веб-браузер и Android Studio Доступ к GitHub Загрузите стартовый код Чтобы начать работу, загрузите стартовый код:

[file_downloadСкачать zip](#)

Кроме того, вы можете клонировать репозиторий GitHub для кода:

```
$ git clone
https://github.com/google-developer-training/basic-android-kotlin-compose-
training-reply-app.git
$ cd basic-android-kotlin-compose-training-reply-app
$ git checkout nav-update
```

2. Предварительные просмотры для разных размеров экрана

Создание предварительных просмотров для разных размеров экрана В коделабе «Создание адаптивного приложения с динамической навигацией» вы узнали, как использовать композитные элементы предварительного просмотра для облегчения процесса разработки. Для адаптивного

приложения лучше всего создавать несколько предварительных просмотров, чтобы показать приложение на разных размерах экрана. Благодаря нескольким предварительным просмотрам вы сможете увидеть свои изменения сразу на всех размерах экрана. Более того, предварительные просмотры также служат документацией для других разработчиков, которые просматривают ваш код, чтобы убедиться в совместимости приложения с разными размерами экрана.

Ранее у вас было только одно превью, поддерживающее компактный экран. Далее вы добавите другие превью.

Чтобы добавить предварительный просмотр для среднего и расширенного экранов, выполните следующие действия:

Добавьте предварительный просмотр для средних экранов, задав значение `medium widthDp` в параметре аннотации `Preview` и указав значение `WindowWidthSizeClass.Medium` в качестве параметра для композита `ReplyApp`. `MainActivity.kt`

```
...
@Preview(showBackground = true, widthDp = 700)
@Composable
fun ReplyAppMediumPreview() {
    ReplyTheme {
        Surface {
            ReplyApp(windowSize = WindowWidthSizeClass.Medium)
        }
    }
}
...
```

Добавьте еще один предварительный просмотр для расширенных экранов, задав большое значение `widthDp` в параметре аннотации `Preview` и указав значение `WindowWidthSizeClass.Expanded` в качестве параметра для композита `ReplyApp`. `MainActivity.kt`

```
...
@Preview(showBackground = true, widthDp = 1000)
@Composable
fun ReplyAppExpandedPreview() {
    ReplyTheme {
        Surface {
            ReplyApp(windowSize = WindowWidthSizeClass.Expanded)
        }
    }
}
...
```

Постройте предварительный просмотр, чтобы увидеть следующее:

Build a preview to see the following:

Вы можете улучшить этот макет, применив один из канонических макетов. Канонические макеты - это композиции на большом экране, которые служат отправной точкой для проектирования и реализации. Вы можете использовать три доступных макета, чтобы определить, как организовать общие элементы в приложении: просмотр списка, вспомогательная панель и лента. В каждом макете учитываются общие сценарии использования и компоненты, чтобы учесть ожидания и потребности пользователей в том, как приложения адаптируются к разным размерам экрана и точкам разрыва.

Для приложения «Ответить» давайте реализуем вид «список-детали», поскольку он лучше всего подходит для просмотра контента и быстрого ознакомления с деталями. При использовании макета «список-детали» вы создадите еще одну панель рядом с экраном списка писем для отображения подробной информации о письме. Такой макет позволяет использовать доступный экран для отображения дополнительной информации и сделать ваше приложение более продуктивным.

Реализация представления списка с деталями Чтобы реализовать представление списка-деталей для расширенных экранов, выполните следующие действия:

Для представления различных типов компоновки контента в `WindowStateUtils.kt` создайте новый класс Enum для различных типов контента. Используйте значение `LIST_AND_DETAIL` для случаев, когда используется расширенный экран, и `LIST_ONLY` в остальных случаях. `WindowStateUtils.kt`

```
...
enum class ReplyContentType {
    LIST_ONLY, LIST_AND_DETAIL
}
...
```

Объявите переменную `contentType` в `ReplyApp.kt` и присвойте ей соответствующий `contentType` для различных размеров окна, чтобы помочь определить подходящий выбор типа содержимого в зависимости от размера экрана. `ReplyApp.kt`

```
...
import com.example.reply.ui.utils.ReplyContentType
...

val navigationType: ReplyNavigationType
val contentType: ReplyContentType

when (windowSize) {
    WindowWidthSizeClass.Compact -> {
        ...
        contentType = ReplyContentType.LIST_ONLY
    }
    WindowWidthSizeClass.Medium -> {
```

```

        ...
        contentType = ReplyContentType.LIST_ONLY
    }
    WindowWidthSizeClass.Expanded -> {
        ...
        contentType = ReplyContentType.LIST_AND_DETAIL
    }
    else -> {
        ...
        contentType = ReplyContentType.LIST_ONLY
    }
}
...

```

Далее вы можете использовать значение `contentType` для создания различных ветвлений для макетов в составном элементе `ReplyAppContent`.

В файле `ReplyHomeScreen.kt` добавьте `contentType` в качестве параметра к композиту `ReplyHomeScreen`. `ReplyHomeScreen.kt`

```

...
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ReplyHomeScreen(
    navigationType: ReplyNavigationType,
    contentType: ReplyContentType,
    replyUiState: ReplyUiState,
    onTabPressed: (MailboxType) -> Unit,
    onEmailCardPressed: (Email) -> Unit,
    onDetailScreenBackPressed: () -> Unit,
    modifier: Modifier = Modifier
) {
    ...

```

Передайте значение `contentType` композиту `ReplyHomeScreen`. `ReplyApp.kt`

```

...
ReplyHomeScreen(
    navigationType = navigationType,
    contentType = contentType,
    replyUiState = replyUiState,
    onTabPressed = { mailboxType: MailboxType ->
        viewModel.updateCurrentMailbox(mailboxType = mailboxType)
        viewModel.resetHomeScreenStates()
    },
    onEmailCardPressed = { email: Email ->
        viewModel.updateDetailsScreenStates(
            email = email
        )
    }
)

```

```

        },
        onDetailScreenBackPressed = {
            viewModel.resetHomeScreenStates()
        },
        modifier = modifier
    )

    ...

```

Добавьте `contentType` в качестве параметра для композита `ReplyAppContent`. `ReplyHomeScreen.kt`

```

...
@Composable
private fun ReplyAppContent(
    navigationType: ReplyNavigationType,
    contentType: ReplyContentType,
    replyUiState: ReplyUiState,
    onTabPressed: ((MailboxType) -> Unit),
    onEmailCardPressed: (Email) -> Unit,
    navigationItemContentList: List<NavigationItemContent>,
    modifier: Modifier = Modifier
) {
    ...

```

Передайте значение `contentType` двум составным элементам `ReplyAppContent`. `ReplyHomeScreen.kt`

```

...
        ReplyAppContent(
            navigationType = navigationType,
            contentType = contentType,
            replyUiState = replyUiState,
            onTabPressed = onTabPressed,
            onEmailCardPressed = onEmailCardPressed,
            navigationItemContentList = navigationItemContentList,
            modifier = modifier
        )
    }
} else {
    if (replyUiState.isShowingHomepage) {
        ReplyAppContent(
            navigationType = navigationType,
            contentType = contentType,
            replyUiState = replyUiState,
            onTabPressed = onTabPressed,
            onEmailCardPressed = onEmailCardPressed,
            navigationItemContentList = navigationItemContentList,
            modifier = modifier
        )
    } else {

```

```

        ReplyDetailsScreen(
            replyUiState = replyUiState,
            isFullScreen = true,
            onBackButtonClicked = onDetailScreenBackPressed,
            modifier = modifier
        )
    }
}
...

```

Давайте отобразим либо полный список и подробный экран, когда значение `contentType` равно `LIST_AND_DETAIL`, либо содержимое только списка, когда значение `contentType` равно `LIST_ONLY`.

В `ReplyHomeScreen.kt` добавьте оператор `if/else` в составной части `ReplyAppContent`, чтобы отобразить составную часть `ReplyListAndDetailContent`, когда значение `contentType` равно `LIST_AND_DETAIL`, и отобразить составную часть `ReplyListOnlyContent` в ветке `else`. `ReplyHomeScreen.kt`

```

...
    Column(
        modifier = modifier
        .fillMaxSize()
        .background(MaterialTheme.colorScheme.inverseOnSurface)
    ) {
        if (contentType == ReplyContentType.LIST_AND_DETAIL) {
            ReplyListAndDetailContent(
                replyUiState = replyUiState,
                onEmailCardPressed = onEmailCardPressed,
                modifier = Modifier.weight(1f)
            )
        } else {
            ReplyListOnlyContent(
                replyUiState = replyUiState,
                onEmailCardPressed = onEmailCardPressed,
                modifier = Modifier.weight(1f)
                .padding(
                    horizontal =
dimensionResource(R.dimen.email_list_only_horizontal_padding)
                )
            )
        }
        AnimatedVisibility(visible = navigationType ==
ReplyNavigationType.BOTTOM_NAVIGATION) {
            ReplyBottomNavigationBar(
                currentTab = replyUiState.currentMailbox,
                onTabPressed = onTabPressed,
                navigationItemContentList = navigationItemContentList
            )
        }
    }
}
...

```

Удалите условие `replyUiState.isShowingHomepage`, чтобы показать постоянный навигационный ящик, поскольку пользователю не нужно переходить к детальному представлению, если он использует расширенное представление. `ReplyHomeScreen.kt`

```
...
    if (navigationType == ReplyNavigationType.PERMANENT_NAVIGATION_DRAWER) {
        PermanentNavigationDrawer(
            drawerContent = {

PermanentDrawerSheet(Modifier.width(dimensionResource(R.dimen.drawer_width))) {
                NavigationDrawerContent(
                    selectedDestination = replyUiState.currentMailbox,
                    onTabPressed = onTabPressed,
                    navigationItemContentList = navigationItemContentList,
                    modifier = Modifier
                        .wrapContentWidth()
                        .fillMaxHeight()

                .background(MaterialTheme.colorScheme.inverseOnSurface)

                .padding(dimensionResource(R.dimen.drawer_padding_content))
            )
        }
    }
) {
...

```

Запустите приложение в режиме планшета, чтобы увидеть экран ниже:

Улучшение элементов пользовательского интерфейса для просмотра списка с деталями В настоящее время ваше приложение отображает панель подробностей на главном экране для расширенных экранов.

Однако этот экран содержит лишние элементы, такие как кнопка «Назад», заголовок темы и дополнительные отступы, поскольку он был разработан для отдельного экрана подробностей. Это можно улучшить с помощью простой настройки.

Чтобы улучшить экран подробностей для расширенного просмотра, выполните следующие действия:

В файле `ReplyDetailsScreen.kt` добавьте переменную `isFullScreen` в качестве булевского параметра к составному экрану `ReplyDetailsScreen`.

Это добавление позволяет различать составной экран при его использовании в качестве самостоятельного и при его использовании внутри главного экрана.

`ReplyDetailsScreen.kt`

```
...
@Composable
```

```
fun ReplyDetailsScreen(
    replyUiState: ReplyUiState,
    onBackPressed: () -> Unit,
    modifier: Modifier = Modifier,
    isFullScreen: Boolean = false
) {
    ...
}
```

Внутри композита ReplyDetailsScreen оберните композит ReplyDetailsScreenTopBar оператором if, чтобы он отображался только при полноэкранном режиме работы приложения. ReplyDetailsScreen.kt

```
...
LazyColumn(
    modifier = modifier
        .fillMaxSize()
        .background(color = MaterialTheme.colorScheme.inverseOnSurface)
        .padding(top =
dimensionResource(R.dimen.detail_card_list_padding_top))
) {
    item {
        if (isFullScreen) {
            ReplyDetailsScreenTopBar(
                onBackPressed,
                replyUiState,
                Modifier
                    .fillMaxWidth()
                    .padding(bottom =
dimensionResource(R.dimen.detail_topbar_padding_bottom))
            )
        }
    }
}
...
}
```

Теперь вы можете добавить подложку. Подкладки, необходимые для композита ReplyEmailDetailsCard, отличаются в зависимости от того, используете ли вы его в полноэкранном режиме или нет. Когда вы используете ReplyEmailDetailsCard с другими композитными элементами в расширенном экране, добавляется дополнительная прокладка от других композитных элементов.

Передайте значение isFullScreen композиту ReplyEmailDetailsCard. Передайте модификатор с горизонтальной прокладкой R.dimen.detail_card_outer_padding_horizontal, если экран полноэкранный, и передайте модификатор с конечной прокладкой R.dimen.detail_card_outer_padding_horizontal в противном случае. ReplyDetailsScreen.kt

```
...
    item {
        if (isFullScreen) {
            ReplyDetailsScreenTopBar(
```



```

        onBackPressed,
        replyUiState,
        Modifier
            .fillMaxWidth()
            .padding(bottom =
dimensionResource(R.dimen.detail_topbar_padding_bottom))
        )
    )
}
ReplyEmailDetailsCard(
    email = replyUiState.currentSelectedEmail,
    mailboxType = replyUiState.currentMailbox,
    isFullScreen = isFullScreen,
    modifier = if (isFullScreen) {
        Modifier.padding(horizontal =
dimensionResource(R.dimen.detail_card_outer_padding_horizontal))
    } else {
        Modifier.padding(end =
dimensionResource(R.dimen.detail_card_outer_padding_horizontal))
    }
)
}
...

```

Добавьте значение `isFullScreen` в качестве параметра к композиту `ReplyEmailDetailsCard`.
`ReplyDetailsScreen.kt`

```

...
@OptIn(ExperimentalMaterial3Api::class)
@Composable
private fun ReplyEmailDetailsCard(
    email: Email,
    mailboxType: MailboxType,
    modifier: Modifier = Modifier,
    isFullScreen: Boolean = false
) {
    ...
}

```

Внутри составной карточки `ReplyEmailDetailsCard` отображайте текст темы письма только тогда, когда приложение не находится в полноэкранном режиме, поскольку в полноэкранном макете тема письма уже отображается в качестве заголовка. Если приложение работает в полноэкранном режиме, добавьте разделитель с высотой `R.dimen.detail_content_padding_top`. `ReplyDetailsScreen.kt`

```

...
Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding(dimensionResource(R.dimen.detail_card_inner_padding))
) {

```

```

        DetailsScreenHeader(
            email,
            Modifier.fillMaxWidth()
        )
        if (isFullScreen) {
            Spacer(modifier =
Modifier.height(dimensionResource(R.dimen.detail_content_padding_top)))
        } else {
            Text(
                text = stringResource(email.subject),
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.outline,
                modifier = Modifier.padding(
                    top = dimensionResource(R.dimen.detail_content_padding_top),
                    bottom =
dimensionResource(R.dimen.detail_expanded_subject_body_spacing)
                ),
            )
        }
        Text(
            text = stringResource(email.body),
            style = MaterialTheme.typography.bodyLarge,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
        )
        DetailsScreenButtonBar(mailboxType, displayToast)
    }

    ...

```

В файле ReplyHomeScreen.kt, внутри составного экрана ReplyHomeScreen, передайте значение true для параметра isFullScreen при создании составного экрана ReplyDetailsScreen как самостоятельного. ReplyHomeScreen.kt

```

...
        } else {
            ReplyDetailsScreen(
                replyUiState = replyUiState,
                isFullScreen = true,
                onBackPressed = onDetailScreenBackPressed,
                modifier = modifier
            )
        }
    }
    ...

```

Запустите приложение в режиме планшета и увидите следующий макет:

Настройте обратную обработку для просмотра списка с деталями При использовании расширенных экранов вам не нужно переходить к экрану ReplyDetailsScreen. Вместо этого вы хотите, чтобы приложение закрывалось, когда пользователь выбирает кнопку «Назад». Для этого необходимо настроить обработчик возврата.

Измените обработчик возврата, передав функцию `activity.finish()` в качестве параметра `onBackPressed` составного экрана `ReplyDetailsScreen` внутри составного списка `ReplyListAndDetailContent`.

`ReplyHomeContent.kt`

```
...
import android.app.Activity
import androidx.compose.ui.platform.LocalContext
...
    val activity = LocalContext.current as Activity
    ReplyDetailsScreen(
        replyUiState = replyUiState,
        modifier = Modifier.weight(1f),
        onBackPressed = { activity.finish() }
    )
...
```

4. Проверка для разных размеров экрана

Рекомендации по качеству приложений для больших экранов Чтобы создать отличный и стабильный опыт для пользователей Android, важно создавать и тестировать приложение с учетом качества. Вы можете обратиться к руководству по качеству приложений Core, чтобы определить, как улучшить качество вашего приложения.

Чтобы создать приложение отличного качества для всех форм-факторов, ознакомьтесь с рекомендациями по качеству приложений для больших экранов. Ваше приложение также должно соответствовать требованиям уровня 3 - «Готовность к работе на большом экране».

Вручную протестируйте приложение на готовность к работе на большом экране В руководстве по качеству приложений приведены рекомендации по тестированию устройств и процедуры проверки качества приложения. Давайте рассмотрим пример теста, относящийся к приложению Reply.

Приведенное выше руководство по качеству приложений требует, чтобы приложение сохраняло или восстанавливало свое состояние после изменения конфигурации. Руководство также содержит инструкции о том, как тестировать приложения, как показано на следующем рисунке:

Чтобы вручную проверить приложение Reply на непрерывность конфигурации, выполните следующие действия:

Запустите приложение Reply на устройстве среднего размера или, если вы используете эмулятор с изменяемым размером, в режиме раскладывания. Убедитесь, что для параметра Автоповорот на эмуляторе установлено значение Вкл.

Прокрутите список адресов электронной почты вниз.

Нажмите на карточку электронного письма. Например, откройте письмо от Али.

Поверните устройство, чтобы убедиться, что выбранное письмо по-прежнему соответствует письму, выбранному в книжной ориентации. В этом примере по-прежнему отображается электронное письмо от Али.

Поверните устройство обратно в книжную ориентацию, чтобы проверить, что приложение по-прежнему отображает то же самое письмо.

5. Добавьте автоматизированный тест для адаптивных приложений

Настройте тест для компактного размера экрана В коделабе «Тестирование приложения Cursake» вы научились создавать тесты пользовательского интерфейса. Теперь давайте узнаем, как создавать специальные тесты для разных размеров экрана.

В приложении Reply вы используете разные элементы навигации для разных размеров экрана. Например, вы ожидаете увидеть постоянный навигационный ящик, когда пользователь видит расширенный экран. Полезно создать тесты для проверки существования различных элементов навигации, таких как нижняя навигация, навигационный рельс и навигационный ящик, для разных размеров экрана.

Чтобы создать тест для проверки существования нижнего навигационного элемента на компактном экране, выполните следующие действия:

В каталоге test создайте новый класс Kotlin с именем ReplyAppTest.kt. В классе ReplyAppTest создайте тестовое правило с помощью createAndroidComposeRule, передав в качестве параметра типа ComponentActivity. ComponentActivity используется для доступа к пустой активности вместо MainActivity. ReplyAppTest.kt

```
...
class ReplyAppTest {

    @get:Rule
    val composeTestRule = createAndroidComposeRule<ComponentActivity>()

    ...
}
```

Чтобы различать элементы навигации на экранах, добавьте тег `testTag` в составной панели `ReplyBottomNavigationBar`.

Определите строковый ресурс для `Navigation Bottom`. `strings.xml`

```
..
<resources>
...
  <string name="navigation_bottom">Navigation Bottom</string>
...
</resources>
```

Добавьте строковое имя в качестве аргумента `testTag` для метода `testTag` модификатора в составной панели `ReplyBottomNavigationBar`.

`ReplyHomeScreen.kt`

```
...
val bottomNavigationContentDescription =
stringResource(R.string.navigation_bottom)
ReplyBottomNavigationBar(
  ...
  modifier = Modifier
    .fillMaxWidth()
    .testTag(bottomNavigationContentDescription)
)
...
```

В классе `ReplyAppTest` создайте тестовую функцию для проверки компактного размера экрана. Установите содержимое `composeTestRule` с композитом `ReplyApp` и передайте `WindowWidthSizeClass.Compact` в качестве аргумента `windowSize`.

Примечание: Добавление композита, принимающего `WindowWidthSizeClass` в качестве аргумента, является хорошей практикой для создания тестируемого кода.

`ReplyAppTest.kt`

```
...
@Test
fun compactDevice_verifyUsingBottomNavigation() {
  // Set up compact window
  composeTestRule.setContent {
    ReplyApp(
      windowSize = WindowWidthSizeClass.Compact
    )
  }
}
```

Убедитесь, что нижний элемент навигации существует с тестовым тегом. Вызовите функцию расширения `onNodeWithTagForStringId` в `composeTestRule`, передайте нижнюю строку навигации и вызовите метод `assertExists()`. `ReplyAppTest.kt`

```
...
@Test
fun compactDevice_verifyUsingBottomNavigation() {
    // Set up compact window
    composeTestRule.setContent {
        ReplyApp(
            windowSize = WindowWidthSizeClass.Compact
        )
    }
    // Bottom navigation is displayed
    composeTestRule.onNodeWithTagForStringId(
        R.string.navigation_bottom
    ).assertExists()
}
```

Запустите тест и убедитесь, что он прошел. Настройка теста для среднего и увеличенного размеров экрана Теперь, когда вы успешно создали тест для компактного экрана, давайте создадим соответствующие тесты для среднего и расширенного экранов.

Чтобы создать тесты, проверяющие наличие навигационного рельса и постоянного навигационного ящика для среднего и расширенного экранов, выполните следующие действия:

Определите строковый ресурс для навигационного рельса, чтобы позже использовать его в качестве тестового тега. `strings.xml`

```
...
<resources>
...
    <string name="navigation_rail">Navigation Rail</string>
...
</resources>
```

Передайте строку в качестве тестового тега через модификатор в композите `PermanentNavigationDrawer`. `ReplyHomeScreen.kt`

```
...
val navigationDrawerContentDescription =
    stringResource(R.string.navigation_drawer)
    PermanentNavigationDrawer(
...
        modifier = Modifier.testTag(navigationDrawerContentDescription)
    )
...

```

Передайте строку в качестве тестового тега через модификатор в композите ReplyNavigationRail.
ReplyHomeScreen.kt

```
...
val navigationRailContentDescription = stringResource(R.string.navigation_rail)
ReplyNavigationRail(
    ...
    modifier = Modifier
        .testTag(navigationRailContentDescription)
)
...
```

Добавьте тест, проверяющий наличие элемента навигационного рельса на средних экранах.
ReplyAppTest.kt

```
...
@Test
fun mediumDevice_verifyUsingNavigationRail() {
    // Set up medium window
    composeTestRule.setContent {
        ReplyApp(
            windowSize = WindowWidthSizeClass.Medium
        )
    }
    // Navigation rail is displayed
    composeTestRule.onNodeWithTagForStringId(
        R.string.navigation_rail
    ).assertExists()
}
```

Добавьте тест, проверяющий наличие элемента навигационного ящика в развернутых экранах.
ReplyAppTest.kt

```
...
@Test
fun expandedDevice_verifyUsingNavigationDrawer() {
    // Set up expanded window
    composeTestRule.setContent {
        ReplyApp(
            windowSize = WindowWidthSizeClass.Expanded
        )
    }
    // Navigation drawer is displayed
    composeTestRule.onNodeWithTagForStringId(
        R.string.navigation_drawer
    ).assertExists()
}
```

Для запуска теста используйте эмулятор планшета или эмулятор с изменяемым размером в режиме планшета. Выполните все тесты и убедитесь, что они пройдены. Тест на изменение конфигурации на компактном экране Изменение конфигурации - это обычное явление, которое происходит в жизненном цикле вашего приложения. Например, когда вы меняете ориентацию с книжной на альбомную, происходит изменение конфигурации. Когда происходит изменение конфигурации, важно проверить, что ваше приложение сохраняет свое состояние. Далее вы создадите тесты, имитирующие изменение конфигурации, чтобы проверить, сохраняет ли ваше приложение свое состояние на компактном экране.

Чтобы протестировать изменение конфигурации в компактном экране:

В директории test создайте новый Kotlin-класс ReplyAppStateRestorationTest.kt. В классе ReplyAppStateRestorationTest создайте правило тестирования, используя createAndroidComposeRule и передав ComponentActivity в качестве параметра типа. ReplyAppStateRestorationTest.kt

```
...
class ReplyAppStateRestorationTest {

    /**
     * Note: To access to an empty activity, the code uses ComponentActivity
    instead of
     * MainActivity.
     */
    @get:Rule
    val composeTestRule = createAndroidComposeRule<ComponentActivity>()
}
...
```

Создайте тестовую функцию для проверки того, что электронная почта по-прежнему выбрана на компактном экране после изменения конфигурации.

ReplyAppStateRestorationTest.kt

```
...
@Test
fun compactDevice_selectedEmailEmailRetained_afterConfigChange() {

}
...
```

Чтобы протестировать изменение конфигурации, вам нужно использовать StateRestorationTester.

Настройте StateRestorationTester, передав в качестве аргумента StateRestorationTester правило composeTestRule. Используйте setContent() с композитом ReplyApp и передайте WindowWidthSizeClass.Compact в качестве аргумента windowSize. ReplyAppStateRestorationTest.kt


```

...
@Test
fun compactDevice_selectedEmailEmailRetained_afterConfigChange() {
    // Setup compact window
    val stateRestorationTester = StateRestorationTester(composeTestRule)
    stateRestorationTester.setContent { ReplyApp(windowSize =
WindowWidthSizeClass.Compact) }

}
...

```

Убедитесь, что третье письмо отображается в приложении. Используйте метод `assertIsDisplayed()` в правиле `composeTestRule`, который ищет текст третьего письма. `ReplyAppStateRestorationTest.kt`

```

...
@Test
fun compactDevice_selectedEmailEmailRetained_afterConfigChange() {
    // Setup compact window
    val stateRestorationTester = StateRestorationTester(composeTestRule)
    stateRestorationTester.setContent { ReplyApp(windowSize =
WindowWidthSizeClass.Compact) }

    // Given third email is displayed
    composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)
    ).assertIsDisplayed()
}
...

```

Перейдите к экрану подробностей письма, щелкнув по теме письма. Для перехода используйте метод `performClick()`. `ReplyAppStateRestorationTest.kt`

```

...
@Test
fun compactDevice_selectedEmailEmailRetained_afterConfigChange() {
    // Setup compact window
    val stateRestorationTester = StateRestorationTester(composeTestRule)
    stateRestorationTester.setContent { ReplyApp(windowSize =
WindowWidthSizeClass.Compact) }

    // Given third email is displayed
    composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)
    ).assertIsDisplayed()

    // Open detailed page

```

```

        composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].subject)
        ).performClick()
    }
    ...

```

Убедитесь, что третье письмо отображается на экране подробностей. Убедитесь в существовании кнопки «Назад», чтобы подтвердить, что приложение находится на экране подробностей, и проверьте, что текст третьего письма отображается. ReplyAppStateRestorationTest.kt

```

...
@Test
fun compactDevice_selectedEmailEmailRetained_afterConfigChange() {
    ...
    // Open detailed page
    composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].subject)
        ).performClick()

    // Verify that it shows the detailed screen for the correct email
    composeTestRule.onNodeWithContentDescriptionForStringId(
        R.string.navigation_back
    ).assertExists()
    composeTestRule.onNodeWithText(
}
...

```

Имитируйте изменение конфигурации с помощью
stateRestorationTester.emulateSavedInstanceStateRestore(). ReplyAppStateRestorationTest.kt

```

...
@Test
fun compactDevice_selectedEmailEmailRetained_afterConfigChange() {
    ...
    // Verify that it shows the detailed screen for the correct email
    composeTestRule.onNodeWithContentDescriptionForStringId(
        R.string.navigation_back
    ).assertExists()
    composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)
        ).assertExists()

    // Simulate a config change
    stateRestorationTester.emulateSavedInstanceStateRestore()
}
...

```

Еще раз убедитесь, что третье письмо отображается на экране подробностей. Убедитесь в существовании кнопки «Назад», чтобы подтвердить, что приложение находится на экране подробностей, и проверьте, что текст третьего письма отображается. ReplyAppStateRestorationTest.kt

```
...
@Test
fun compactDevice_selectedEmailEmailRetained_afterConfigChange() {
    // Setup compact window
    val stateRestorationTester = StateRestorationTester(composeTestRule)
    stateRestorationTester.setContent { ReplyApp(windowSize =
WindowWidthSizeClass.Compact) }

    // Given third email is displayed
    composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)
    ).assertIsDisplayed()

    // Open detailed page
    composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].subject)
    ).performClick()

    // Verify that it shows the detailed screen for the correct email
    composeTestRule.onNodeWithContentDescriptionForStringId(
        R.string.navigation_back
    ).assertExists()
    composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)
    ).assertExists()

    // Simulate a config change
    stateRestorationTester.emulateSavedInstanceStateRestore()

    // Verify that it still shows the detailed screen for the same email
    composeTestRule.onNodeWithContentDescriptionForStringId(
        R.string.navigation_back
    ).assertExists()
    composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)
    ).assertExists()
}
...
```

Запустите тест с помощью эмулятора телефона или эмулятора с изменяемым размером в режиме телефона. Убедитесь, что тест пройден. Тест на изменение конфигурации в расширенном экране Чтобы проверить изменение конфигурации на расширенном экране, имитируя изменение конфигурации и передавая соответствующий `WindowWidthSizeClass`, выполните следующие действия:

Создайте тестовую функцию для проверки того, что электронная почта по-прежнему выбрана на экране подробностей после изменения конфигурации.

`ReplyAppStateRestorationTest.kt`

```
...
@Test
fun expandedDevice_selectedEmailEmailRetained_afterConfigChange() {

}
...
```

Чтобы протестировать изменение конфигурации, вам нужно использовать `StateRestorationTester`.

Настройте `StateRestorationTester`, передав в качестве аргумента `StateRestorationTester` правило `composeTestRule`. Используйте `setContent()` с композитным `ReplyApp` и передайте `WindowWidthSizeClass.Expanded` в качестве аргумента `windowSize`. `ReplyAppStateRestorationTest.kt`

```
...
@Test
fun expandedDevice_selectedEmailEmailRetained_afterConfigChange() {
    // Setup expanded window
    val stateRestorationTester = StateRestorationTester(composeTestRule)
    stateRestorationTester.setContent { ReplyApp(windowSize =
WindowWidthSizeClass.Expanded) }
}
...
```

Убедитесь, что третье письмо отображается в приложении. Используйте метод `assertIsDisplayed()` в правиле `composeTestRule`, который ищет текст третьего письма. `ReplyAppStateRestorationTest.kt`

```
...
@Test
fun expandedDevice_selectedEmailEmailRetained_afterConfigChange() {
    // Setup expanded window
    val stateRestorationTester = StateRestorationTester(composeTestRule)
    stateRestorationTester.setContent { ReplyApp(windowSize =
WindowWidthSizeClass.Expanded) }

    // Given third email is displayed
    composeTestRule.onNodeWithText(
```

```
composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)
    ).assertIsDisplayed()
}
...
```

Выберите третье письмо на экране подробностей. Для выбора письма используйте метод `performClick()`. `ReplyAppStateRestorationTest.kt`

```
...
@Test
fun expandedDevice_selectedEmailEmailRetained_afterConfigChange() {
    // Setup expanded window
    val stateRestorationTester = StateRestorationTester(composeTestRule)
    stateRestorationTester.setContent { ReplyApp(windowSize =
        WindowWidthSizeClass.Expanded) }

    // Given third email is displayed
    composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)
    ).assertIsDisplayed()

    // Select third email
    composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].subject)
    ).performClick()
    ...
}

...
```

Убедитесь, что на экране подробностей отображается третье письмо, используя `testTag` на экране подробностей и ища текст на его дочерних элементах. Такой подход позволяет убедиться, что текст находится в разделе подробностей, а не в списке писем. `ReplyAppStateRestorationTest.kt`

```
...

@Test
fun expandedDevice_selectedEmailEmailRetained_afterConfigChange() {
    ...
    // Select third email
    composeTestRule.onNodeWithText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].subject)
    ).performClick()

    // Verify that third email is displayed on the details screen
    composeTestRule.onNodeWithTagForStringId(R.string.details_screen).onChildren()
```

```

        .assertAny(hasAnyDescendant(hasText(
composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)))
    )
    ...
}

...

```

Имитируйте изменение конфигурации с помощью

`stateRestorationTester.emulateSavedInstanceStateRestore()`. `ReplyAppStateRestorationTest.kt`

```

...
@Test
fun expandedDevice_selectedEmailEmailRetained_afterConfigChange() {
    ...
    // Verify that third email is displayed on the details screen
    composeTestRule.onNodeWithTagForStringId(R.string.details_screen).onChildren()
        .assertAny(hasAnyDescendant(hasText(
composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)))
    )

    // Simulate a config change
    stateRestorationTester.emulateSavedInstanceStateRestore()
    ...
}
...

```

Убедитесь еще раз, что на экране подробностей отображается третье письмо после изменения конфигурации. `ReplyAppStateRestorationTest.kt`

```

...
@Test
fun expandedDevice_selectedEmailEmailRetained_afterConfigChange() {
    // Setup expanded window
    val stateRestorationTester = StateRestorationTester(composeTestRule)
    stateRestorationTester.setContent { ReplyApp(windowSize =
WindowWidthSizeClass.Expanded) }

    // Given third email is displayed
    composeTestRule.onNodeWithText(
composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)
    ).assertIsDisplayed()

    // Select third email
    composeTestRule.onNodeWithText(

```

```

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].subject)
    ).performClick()

    // Verify that third email is displayed on the details screen
    composeTestRule.onNodeWithTagForStringId(R.string.details_screen).onChildren()
        .assertAny(hasAnyDescendant(hasText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)))
    )

    // Simulate a config change
    stateRestorationTester.emulateSavedInstanceStateRestore()

    // Verify that third email is still displayed on the details screen
    composeTestRule.onNodeWithTagForStringId(R.string.details_screen).onChildren()
        .assertAny(hasAnyDescendant(hasText(

composeTestRule.activity.getString(LocalEmailsDataProvider.allEmails[2].body)))
    )
}
...

```

Запустите тест с помощью эмулятора планшета или эмулятора с изменяемыми размерами в режиме планшета. Убедитесь, что тест пройден. Использование аннотаций для группировки тестов для разных размеров экрана Из предыдущих тестов вы могли понять, что некоторые тесты не работают, когда их запускают на устройствах с несовместимым размером экрана. Хотя вы можете запускать тесты по одному, используя соответствующее устройство, такой подход может не подойти, если у вас много тестовых случаев.

Чтобы решить эту проблему, можно создать аннотации, обозначающие размеры экранов, на которых может работать тест, и настроить аннотированный тест для соответствующих устройств.

Чтобы запустить тест на основе размеров экрана, выполните следующие действия:

В каталоге теста создайте TestAnnotations.kt, который содержит три класса аннотаций: TestCompactWidth, TestMediumWidth, TestExpandedWidth. TestAnnotations.kt

```

...
annotation class TestCompactWidth
annotation class TestMediumWidth
annotation class TestExpandedWidth
...

```

Используйте аннотации к тестовым функциям для компактных тестов, поместив аннотацию TestCompactWidth после аннотации теста для компактного теста в ReplyAppTest и ReplyAppStateRestorationTest. ReplyAppTest.kt

```

...
@Test
@TestCompactWidth
fun compactDevice_verifyUsingBottomNavigation() {
...

```

ReplyAppStateRestorationTest.kt

```

...
@Test
@TestCompactWidth
fun compactDevice_selectedEmailEmailRetained_afterConfigChange() {
...

```

Используйте аннотации к тестовым функциям для средних тестов, поместив аннотацию `TestMediumWidth` после аннотации теста для среднего теста в `ReplyAppTest`. `ReplyAppTest.kt`

```

...
@Test
@TestMediumWidth
fun mediumDevice_verifyUsingNavigationRail() {
...

```

Используйте аннотации к тестовым функциям для расширенных тестов, поместив аннотацию `TestExpandedWidth` после аннотации теста для расширенного теста в `ReplyAppTest` и `ReplyAppStateRestorationTest`. `ReplyAppTest.kt`

```

...
@Test
@TestExpandedWidth
fun expandedDevice_verifyUsingNavigationDrawer() {
...

```

ReplyAppStateRestorationTest.kt

```

...
@Test
@TestExpandedWidth
fun expandedDevice_selectedEmailEmailRetained_afterConfigChange() {
...

```


Чтобы гарантировать успех, настройте тест на выполнение только тех тестов, которые аннотированы с `TestCompactWidth`.

В Android Studio выберите `Run > Edit Configurations...`

Переименуйте тест в `Compact tests` и выберите запуск теста `All in Package`.

Щелкните на трех точках (...) справа от поля `Instrumentation arguments`. Нажмите кнопку с плюсом (+) и добавьте дополнительные параметры: `annotation` со значением `com.example.reply.test.TestCompactWidth`.

Запустите тесты с компактным эмулятором. Убедитесь, что были запущены только компактные тесты.

Повторите действия для среднего и расширенного экранов.

6. Получите код решения

Чтобы загрузить код готового codelab, используйте следующую команду `git`:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-reply-app.git
```

Также вы можете скачать репозиторий в виде zip-файла, разархивировать его и открыть в Android Studio.

7. Заключение Поздравляем! Вы сделали приложение Reply адаптивным для всех размеров экрана, реализовав адаптивный макет. Вы также научились ускорять разработку с помощью предварительных просмотров и поддерживать качество приложения с помощью различных методов тестирования.

Не забудьте поделиться своей работой в социальных сетях с помощью `#AndroidBasics!`